

0.1 Riconoscimento Note

I test delle Sezioni precedenti usano i preset quadratici sintetici dell’applicazione. Per verificare i metodi su un problema di apprendimento reale, li applichiamo al riconoscimento della *nota* suonata (pitch class) sul dataset **NSynth** [1]: un benchmark di riferimento per la ricerca audio, contenente circa 306 000 note monofoniche di quattro secondi (16 kHz) generate da 1006 strumenti di biblioteche campione commerciali, ciascuna annotata con la nota e la famiglia strumentale di appartenenza.

Per contenere il costo del download (lo split di addestramento completo occupa circa 24 GB), usiamo lo split *validation* (12 678 clip) come insieme di addestramento e lo split *test* (4 096 clip) come insieme di valutazione. Per costruzione del dataset gli strumenti dei due split sono disgiunti: il test misura quindi la generalizzazione a strumenti mai incontrati durante l’addestramento.

Features audio. Da ogni clip estraiamo il *chromagramma*: le dodici componenti cromatiche, cioè l’energia in ciascuna delle 12 classi di altezza (C , $C\#$, D , $\dots$, B), indipendenti dall’ottava, e ne prendiamo media e deviazione standard nel tempo, ottenendo 24 features per clip. Il chromagramma è una rappresentazione musicalmente significativa e, essendo per costruzione indipendente dall’ottava, è particolarmente adatto a questo compito.

Problema di apprendimento. Il compito è la classificazione multinomiale in dodici classi di altezza mediante regressione logistica (softmax). La perdita empirica è

$$J(w) = -\frac{1}{N} \sum_{i=1}^N \log p_{y_i}(x_i; w) + \frac{\lambda}{2} \|W\|_F^2,$$

con $p_c(x; w) \propto \exp(e_c^\top Wx)$, dove $W \in \mathbb{R}^{12 \times 25}$ (24 features + bias) e $w = \text{vec}(W)$: 300 parametri in tutto. I metodi a gradiente e Newton usano la regolarizzazione L_2 con $\lambda = 10^{-4}$; il metodo Newton-CG L_1 risolve la versione L_1 -regolarizzata con $\nu = 10^{-3}$ (Sezione 5.3). I parametri degli algoritmi sono quelli della Sezione 6.1: $\theta = 0.5$, $n_0 = 64$, $\alpha = 1$, budget di 300 iterazioni, $R = 0.1$ e seed 42; la dimensione del batch è selezionata dinamicamente dalla CCV.

Il modello è una regressione logistica multinomiale: a ogni clip x (vettore di 24 features) assegna a ogni nota c un punteggio

$$s_c(x) = w_{c,1}x_1 + \dots + w_{c,24}x_{24} + b_c,$$

combinazione lineare delle features, poi convertito in probabilità dalla softmax. È “lineare” perché la decisione è una somma pesata degli input, senza interazioni tra le features (in due dimensioni due classi sarebbero separate da una retta); l’unica non-linearità è la softmax, che serve solo a produrre probabilità. La scelta è deliberata: la loss è convessa e liscia, quindi la teoria della tesi (CCV, convergenza, complessità) si applica direttamente, a differenza delle reti profonde su spettrogrammi (loss non convessa, milioni di parametri). Rispetto a una rete profonda il modello è più semplice: input di 24 statistiche aggregate invece dello spettrogramma e decisione lineare senza features apprese.

Il codice del notebook `nsynth_nota_riproduzione` è organizzato in un piccolo numero di **funzioni** che si richiamano tra loro: la Figura 0.1 ne mostra la mappa completa, mentre le Figure 0.2–0.7 mostrano il dettaglio di ciascuna, (ingressi $\rightarrow$ uscite) e le formule esatte implementate. I nomi coincidono con quelli del notebook, così gli schemi si confrontano direttamente con il codice.

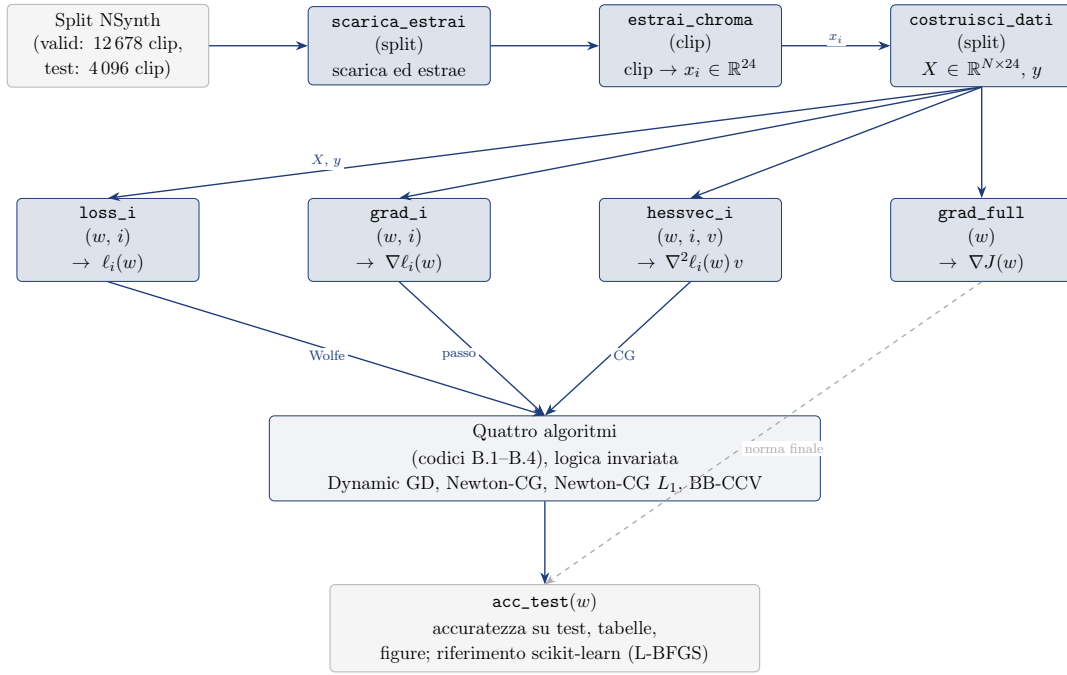


Figura 0.1: Mappa delle funzioni del notebook `nsynth_nota_riproduzione`. Dalle clip di uno split si estraggono i vettori x_i (Figura 0.2), raccolti in X con le etichette delle note da `costruisci_dati`; le quattro funzioni del nucleo forniscono agli algoritmi dei codici B.1–B.4 proprio gli oggetti che essi richiedono: la loss per la line search di Wolfe, il gradiente per il passo, il prodotto Hessiano–vettore per il gradiente coniugato; `grad_full` e `acc_test` servono alle metriche di valutazione. Freccie continue: flusso dei dati; frecce tratteggiate: chiamate tra funzioni.

Estrazione delle features (audio $\rightarrow$ vettore). La funzione `estrai_chroma` converte una clip in un vettore $x_i \in \mathbb{R}^{24}$ di statistiche cromatiche: calcola il chromagramma, cioè l’energia in ciascuna delle 12 classi di altezza in ogni frame temporale, e lo riassume nel tempo con media e deviazione standard (Figura 0.2). Ogni componente misura quindi quanta energia porta una determinata classe di nota, mediata (o la sua variabilità) sull’intera clip.

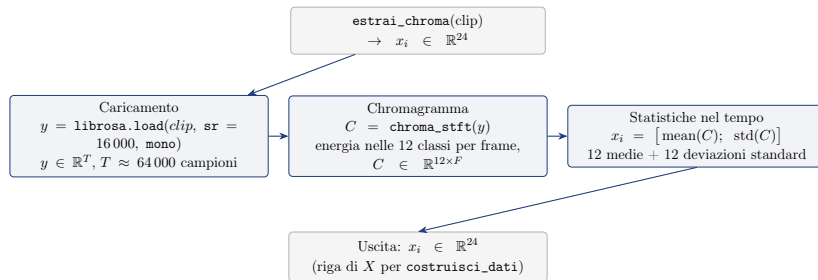


Figura 0.2: Funzione `estrai_chroma`: da clip audio a vettore di 24 statistiche cromatiche. Il chromagramma C misura l’energia in ciascuna delle 12 classi di altezza in ogni frame temporale; media e deviazione standard nel tempo producono il vettore x_i .

Costruzione del dataset. La funzione `costruisci_dati` passa le clip di uno split a `estrai_chroma` e costruisce la matrice X e il vettore delle etichette y : per ogni

clip l'etichetta è la *pitch class* p mod 12 della nota annotata ($0 = C, \dots, 11 = B$). Le colonne sono poi standardizzate con media e deviazione standard calcolate sul solo training set, applicando le stesse statistiche a test (Figura 0.3).

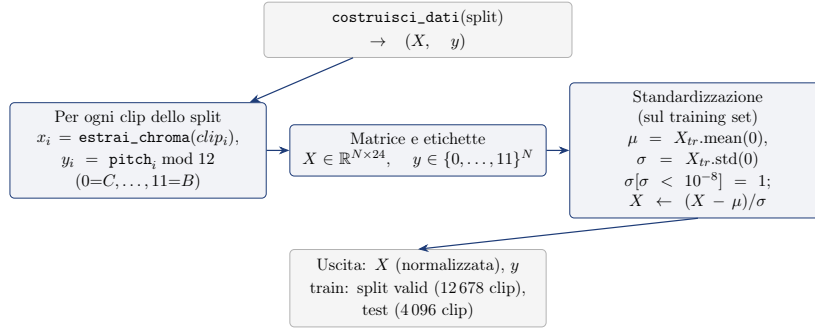


Figura 0.3: Funzione `costruisci_dati`: costruisce la matrice delle features e le etichette della pitch class per uno split, poi le normalizza con media e deviazione standard calcolate sul solo training set (le stesse statistiche sono applicate a test).

Le funzioni per-esempio. I codici B.1–B.4 operano per-esempio: a ogni iterazione richiamano in loop `loss_i`, `grad_i` e `hessvec_i` sul sottocampione corrente, con w fissato. Le tre funzioni condividono una cache sui pesi (le quantità che dipendono solo da w sono ricalcolate una volta sola). Il modello è una regressione logistica multinomiale: la Figura 0.4 mostra la loss di un singolo esempio, la Figura 0.5 il suo gradiente e la Figura 0.6 il prodotto Hessiano–vettore.

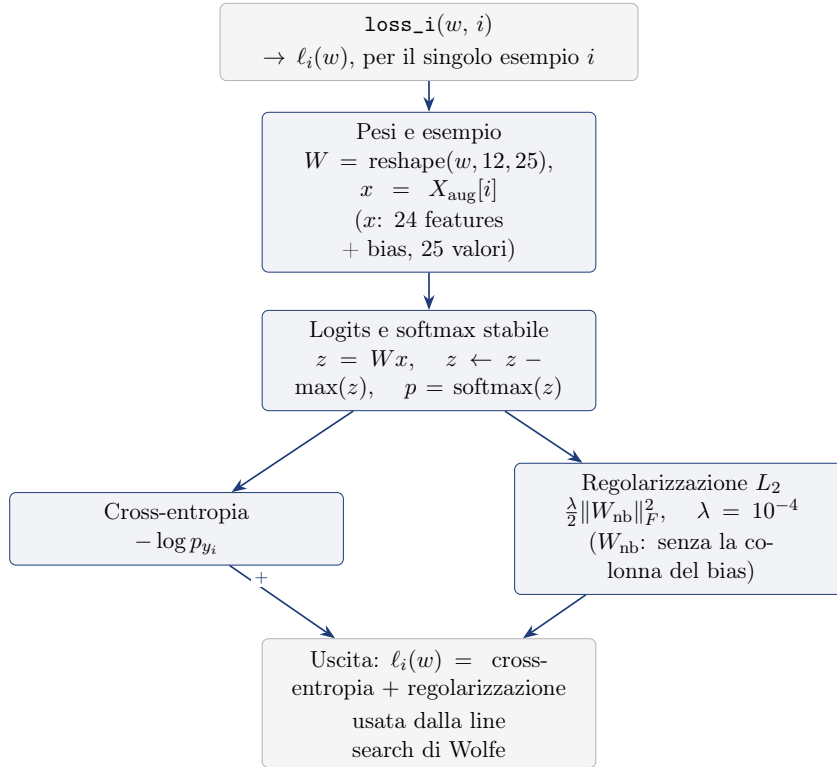


Figura 0.4: Funzione `loss_i`: logits della regressione logistica multinomiale, softmax stabile (shift del massimo), cross-entropia per l'esempio i e regolarizzazione L_2 sui pesi; l'uscita è la somma dei due contributi.

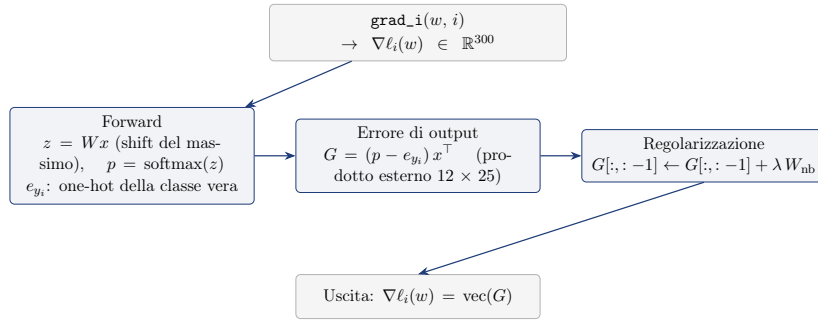


Figura 0.5: Funzione `grad_i`: gradiente per-esempio della regressione logistica. L'errore $p - e_{y_i}$ tra le probabilità e la codifica one-hot della classe vera, esteriorizzato con l'input x , dà la matrice 12×25 ; la colonna del bias (l'ultima) non riceve regolarizzazione.

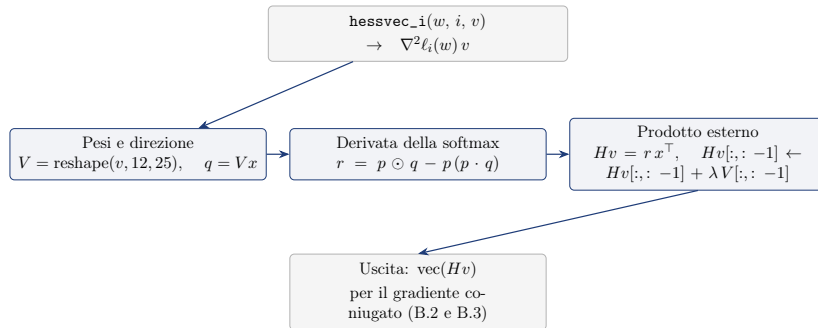


Figura 0.6: Funzione `hessvec_i`: prodotto Hessiano–vettore per-esempio. L'Hessiana della regressione logistica applicata a v richiede la derivata della softmax lungo la direzione $q = Vx$; la regolarizzazione L_2 contribuisce solo sulle colonne dei pesi.

Gradiente sul training set completo. La funzione `grad_full` calcola in forma vettorizzata il gradiente di J su tutto il training set, con la stessa formula di `grad_i` (Figura 0.7). *Vettorizzata* significa che il calcolo non è ripetuto in un ciclo sui singoli esempi (una chiamata a `grad_i` per ciascuna delle N clip), ma eseguito una sola volta su matrici che raccolgono l'intero dataset: un unico prodotto $Z = X_{\text{aug}} W^\top$ dà i logits di tutte le clip, la softmax è applicata per righe e la media $(P - Y)^\top X_{\text{aug}} / N$ somma i contributi in forma matriciale. Il risultato è matematicamente identico alla somma dei contributi individuali, ma le operazioni vettoriali di NumPy delegano i prodotti a kernel di algebra lineare ottimizzati e risultano molto più veloci. La funzione serve per le metriche di valutazione (norma del gradiente finale) e per il subgradiente del metodo L_1 .

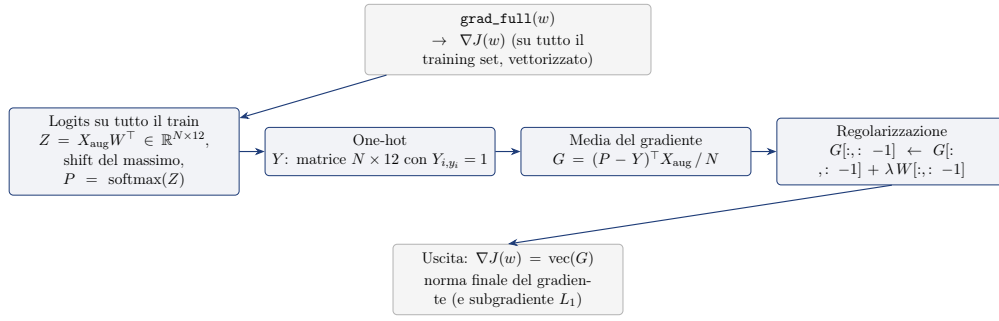


Figura 0.7: Funzione **grad_full**: gradiente di J su tutto il training set in forma matriciale. Le probabilità di tutte le N clip sono calcolate in un unico prodotto; la media degli errori $(P - Y)$ pesati con le features dà il gradiente, usato per la norma finale e per il subgradiente del metodo L_1 .

Abbiamo applicato gli stessi metodi anche al riconoscimento della famiglia strumentale (10 classi, features di mel), dove il modello lineare si ferma a circa il 59% di accuratezza contro il 20.8% della classe maggioritaria: il limite è nella rappresentazione, non negli algoritmi di ottimizzazione. Riportiamo qui il compito del pitch class perché più pulito e direttamente confrontabile con il solutore di riferimento.

I risultati sono riportati nella Tabella 0.1 e nelle Figure 0.8–0.9.

Metodo	Acc. test	$\ \nabla J(w)\ _2$	Batch finale	Tempo (s)	Coeff. non nulli
Dynamic GD	88.0%	6.8×10^{-2}	5 297	7.3	—
Newton-CG	91.4%	2.8×10^{-6}	12 678	6.3	—
Newton-CG L_1	89.8%	8.5×10^{-7}	12 678	1.1	146/300
BB-CCV	90.8%	3.1×10^{-3}	12 678	2.4	—
Riferimento: scikit-learn (L-BFGS) = 91.5%					

Tabella 0.1: Riconoscimento della nota (12 classi di altezza) su NSynth con features chroma (24 dimensioni): accuratezza di test dopo 300 iterazioni di budget e norma del gradiente finale. Newton-CG L_1 raggiunge il criterio di arresto $\|\nabla J\|_2 < 10^{-6}$.

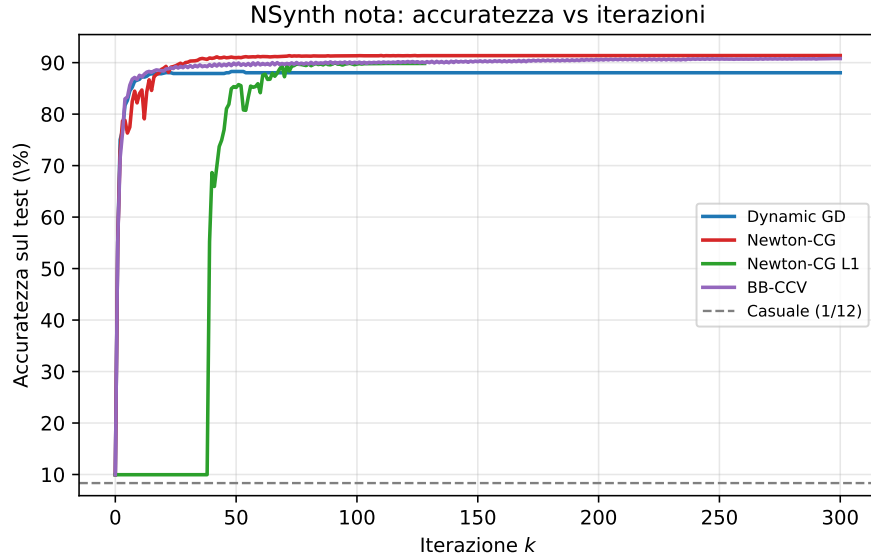


Figura 0.8: Accuratezza sul test NSynth per il riconoscimento della nota in funzione delle iterazioni k . La linea tratteggiata indica la predizione casuale ($1/12 \approx 8.3\%$).

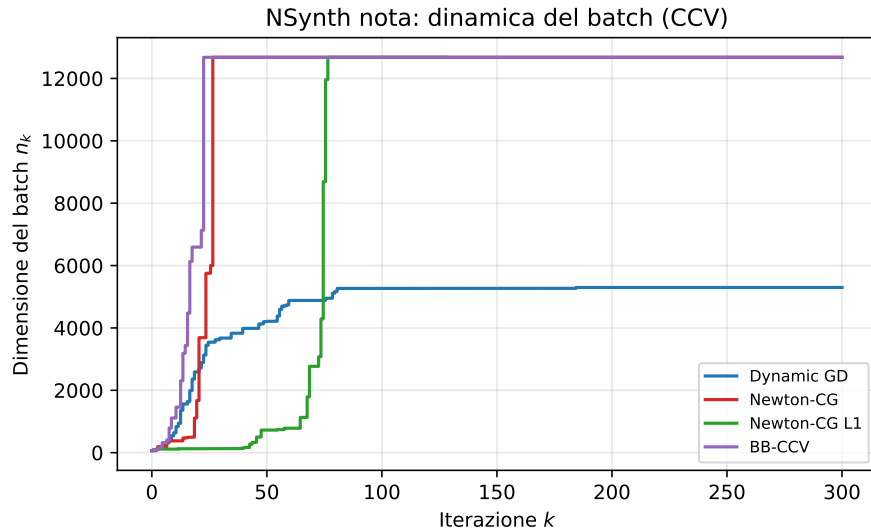


Figura 0.9: Dimensione del batch n_k per il riconoscimento della nota: anche su questo compito la CCV aumenta il campione man mano che il gradiente si riduce.

Tutti i metodi superano di gran lunga la predizione casuale (8.3%): Dynamic GD raggiunge l'88.0%, BB-CCV il 90.8% e Newton-CG il 91.4%, in linea con il solutore di riferimento (L-BFGS di scikit-learn, 91.5%): il metodo del secondo ordine converge quindi allo stesso ottimo del solutore standard, con una differenza inferiore a un decimo di punto percentuale. Il metodo Newton-CG L_1 ($\nu = 10^{-3}$) raggiunge l'89.8% e termina entro il criterio di arresto in 128 iterazioni, azzerando il 51% dei coefficienti (146 restano non nulli su 300): la regolarizzazione L_1 seleziona le componenti cromatiche rilevanti per il riconoscimento della nota. Anche su questo compito la CCV aumenta la dimensione del batch man mano che il gradiente si riduce (Figura 0.9). Il solo Dynamic GD fa eccezione: la norma del suo gradiente

ristagna a $\approx 6.8 \times 10^{-2}$ (non scende mai sotto la tolleranza di arresto) e perciò la CCV si stabilizza a $n_k = 5\,297$ invece di spingere il campione fino a N . In sintesi, gli stessi algoritmi validati sui problemi sintetici raggiungono un’accuratezza elevata ($\approx 91\%$) sul compito musicale del riconoscimento della nota, in linea con il solutore di riferimento.

Riproducibilità. Tutti gli esperimenti di questa sezione (estrazione delle features, esecuzione dei quattro metodi, figure e tabelle) sono riproducibili con il notebook Colab `nsynth_nota_riproduzione`, https://colab.research.google.com/github/Alessandro1040/selezione-dinamica-batch/blob/main/tesi/nsynth/nsynth_nota_riproduzione.ipynb; il codice è anche disponibile nel repository GitHub (<https://github.com/Alessandro1040/selezione-dinamica-batch>), nella cartella `tesi/nsynth/`.

Precisione sui valori numerici. I dati riportati in questa sezione (accuratezze, norme del gradiente, dimensioni del batch e tempi) si riferiscono alla run di riferimento eseguita sul computer dell’autore, non a un’esecuzione in Colab. Il notebook Colab riproduce lo stesso codice con lo stesso seme casuale (seed 42), ma la riproducibilità non è esatta: gli algoritmi sono stocastici e i calcoli in virgola mobile dipendono dall’ambiente di esecuzione (versione di NumPy e libreria BLAS sottostante, OpenBLAS in Colab contro Accelerate su macOS), per cui piccole differenze di arrotondamento, accumulate sulle 300 iterazioni, producono valori finali leggermente diversi. Le differenze sono dell’ordine di qualche decimo di punto percentuale e non cambiano le conclusioni; i tempi di esecuzione in Colab sono invece molto più lunghi (minuti, contro pochi secondi sulla macchina locale).

0.2 Riconoscimento Strumenti

Per verificare come si comportano i quattro metodi quando la funzione obiettivo **non** è convessa, gli stessi algoritmi dei codici B.1–B.4 sono stati applicati a una **rete neurale**, una famiglia di modelli in cui la loss dipende in modo non lineare dai parametri e non è convessa. Le regole degli algoritmi (line search di Wolfe, controllo della varianza CCV, gradiente coniugato, proiezione sull’ortante) restano *invariate*; cambia solo il modo in cui si calcolano gradiente, loss e prodotto Hessiano–vettore. Nelle versioni originali questi oggetti si ottengono sommando, per ogni esempio del batch, il contributo individuale: con una rete di $\approx 234\,000$ parametri e $N = 12\,678$ esempi questo costo (e la memoria necessaria) rende l’esecuzione impraticabile su Colab. Si usano quindi versioni *batch*, dette *vettorizzate*: applicano le stesse formule a matrici che rappresentano l’intero batch in una sola volta, i prodotti $X_b W_1$, $a_1 W_2$ e $a_2 W_3$ del forward e i prodotti matriciali del passaggio all’indietro, invece di ripetere il calcolo in un ciclo per ogni esempio (come definito nella Sezione 0.1). La corrispondenza con le versioni per-esempio è verificata numericamente (errori dell’ordine di 10^{-15}).

L’implementazione è organizzata in un piccolo numero di **funzioni** che si richiamano tra loro: la Figura 0.10 ne mostra la mappa completa, mentre le Figure 0.11–0.16 mostrano il dettaglio di ciascuna, (ingressi $\rightarrow$ uscite) e le formule esatte implementate. I nomi coincidono con quelli del notebook Colab `nsynth_net_riproduzione`, così gli schemi si confrontano direttamente con il codice.

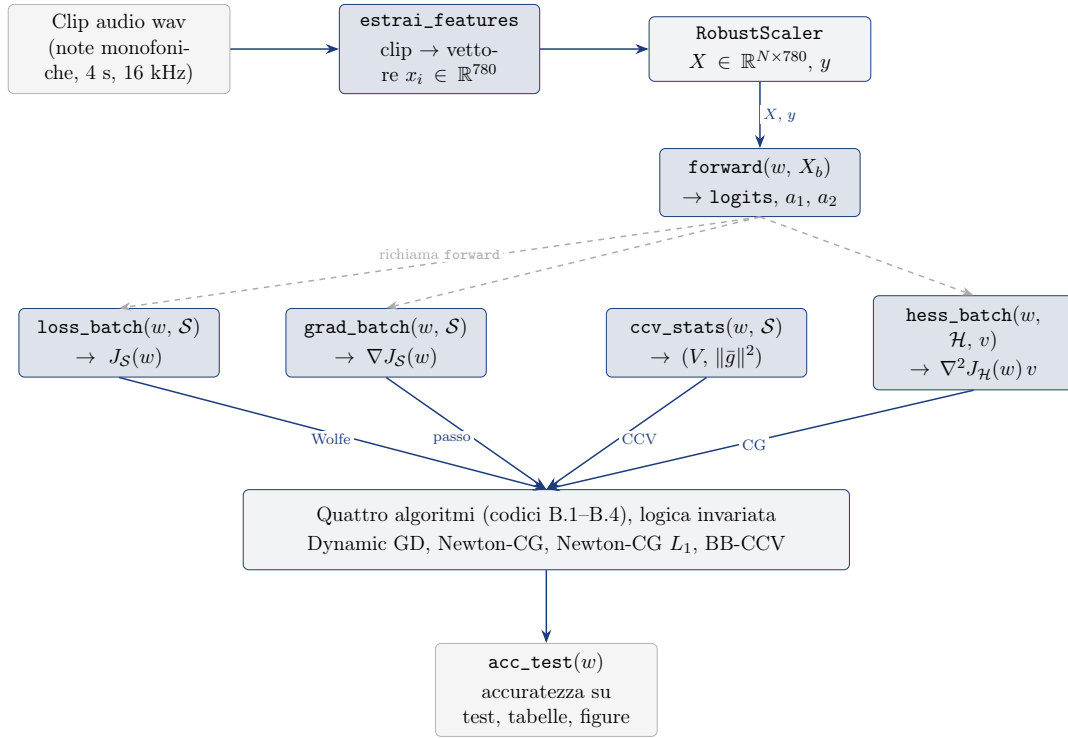


Figura 0.10: Mappa delle funzioni dell’implementazione vettorizzata. Dalle clip si estraggono i vettori x_i (Figura 0.11), normalizzati nella matrice X ; **forward** calcola le attivazioni della rete e le quattro funzioni che la richiamano forniscono agli algoritmi dei codici B.1–B.4 proprio gli oggetti che essi richiedono: la loss per la line search di Wolfe, il gradiente per il passo, la varianza per la verifica CCV e il prodotto Hessiano–vettore per il gradiente coniugato; **acc_test** valuta infine la soluzione sul test. Freccie continue: flusso dei dati; frecce tratteggiate: chiamate tra funzioni.

Estrazione delle features (audio $\rightarrow$ vettore). Una clip è un segnale $y \in \mathbb{R}^T$ di 64 000 campioni a 16 kHz, cioè un vettore di pressioni d’aria campionate, da cui nessun algoritmo lavora direttamente. La funzione **estrai_features** lo converte in un vettore $x_i \in \mathbb{R}^{780}$ di statistiche che riassumono *come suona* la clip: separa prima il segnale in una parte armonica y_h (il tono) e una percussiva y_p (gli attacchi), poi calcola su y_h sei famiglie di descrittori (timbro, altezza, forma spettrale, dinamica) riassumendo ogni descrittore nel tempo con media e deviazione standard (Figura 0.11); i valori sono infine normalizzati con un **RobustScaler**. Un programmatore non deve conoscere l’audio: basta sapere che x_i è l’input della rete e che ogni sua componente misura una caratteristica statistica del suono.

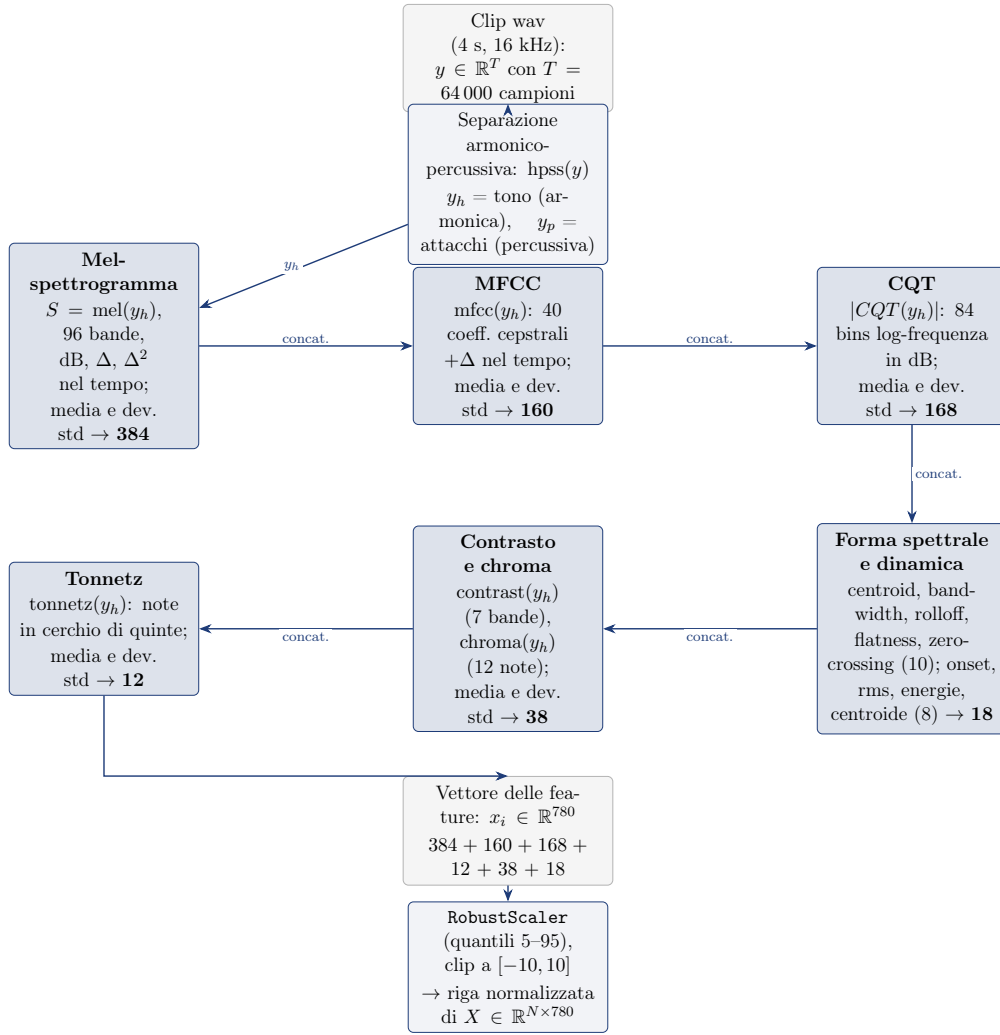


Figura 0.11: `estrai_features`: da clip audio a vettore di feature. Il segnale y è separato in armonica (y_h) e percussiva (y_p); sei famiglie di descrittori calcolati su y_h (tonnetz, spettrale e dinamica anche su y) producono medie e deviazioni standard nel tempo, concatenate nel vettore $x_i \in \mathbb{R}^{780}$ (le cifre indicano le dimensioni di ogni blocco); infine `RobustScaler` normalizza le colonne sui quantili 5–95 e i valori sono limitati a $[-10, 10]$. Le frecce etichettate *concat.* indicano che i blocchi, calcolati indipendentemente, sono concatenati nell'ordine mostrato.

Forward della rete. La rete è una funzione f_w che trasforma il batch $X_b \in \mathbb{R}^{m \times 780}$ in un vettore di punteggi per le 10 famiglie strumentali, alternando trasformazioni affini e attivazioni non lineari $\tanh$: z_l è l'ingresso dello strato l , $a_l = \tanh(z_l)$ la sua uscita, e l'ultimo strato produce i *logits*, che la softmax convertirà in probabilità. I parametri sono raccolti nel vettore $w = [W_1; b_1; W_2; b_2; W_3; b_3]$ di $\approx 234\,000$ valori (Figura 0.12); la funzione restituisce anche le attivazioni a_1, a_2 , che servono alla backpropagation.

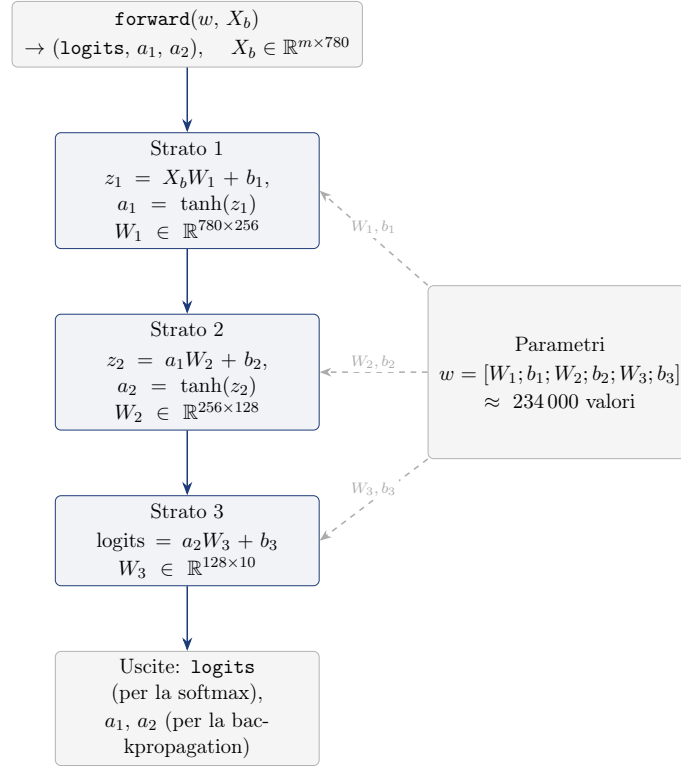


Figura 0.12: Funzione **forward** (architettura della rete): tre trasformazioni affini intervallate dall'attivazione $\tanh$, con le dimensioni delle matrici; w è l'intero vettore dei parametri, da cui ogni strato legge i propri pesi W_l e bias b_l . I logits sono le uscite grezze dell'ultimo strato.

Loss sul batch. La funzione **loss_batch** restituisce la perdita media sul sotto-campione $\mathcal{S}$, $J_{\mathcal{S}}(w) = -\frac{1}{m} \sum_{i \in \mathcal{S}} \log p_{y_i} + \frac{\lambda}{2} \sum_{l=1}^3 \|W_l\|_F^2$: le probabilità p si ottengono con la softmax dai logits della **forward**, e la regolarizzazione L_2 (con $\lambda = 10^{-4}$) è applicata solo alle matrici dei pesi W_l , non ai bias. La Figura 0.13 riassume i passaggi.

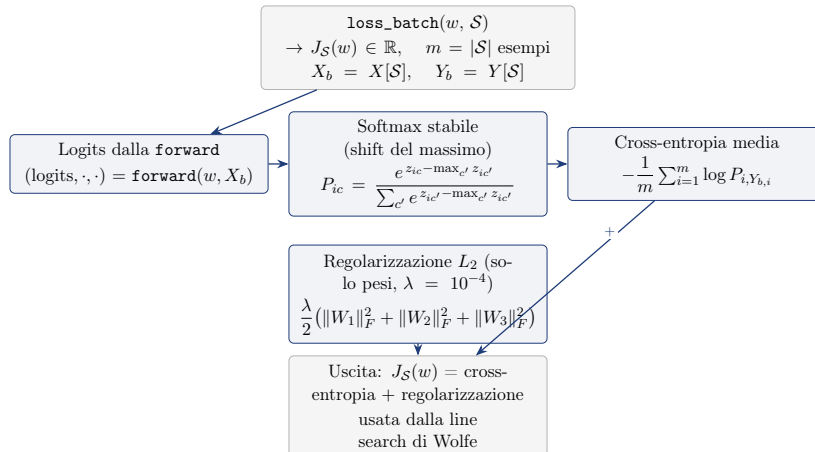


Figura 0.13: Funzione **loss_batch**: logits dalla **forward**, softmax stabile, cross-entropia media sul batch e regolarizzazione L_2 sui pesi; l'uscita è la somma dei due contributi.

Gradiente del batch. Per minimizzare la loss serve il gradiente rispetto ai parametri. Lo si calcola con la **backpropagation**: dopo il forward si definisce l'errore $\delta = (P - Y_b)/m$ tra le probabilità P e la codifica one-hot Y_b delle classi vere, e lo si propaga all'indietro attraverso i tre strati usando la derivata di tanh, che vale $1 - a^2$; a ogni peso si somma il contributo di regolarizzazione λW_l . Il risultato è la media dei gradienti individuali, $\nabla J_S(w) = \frac{1}{m} \sum_{i \in S} \nabla \ell(w; i)$, calcolata però in un solo passaggio matriciale sull'intero batch (Figura 0.14).

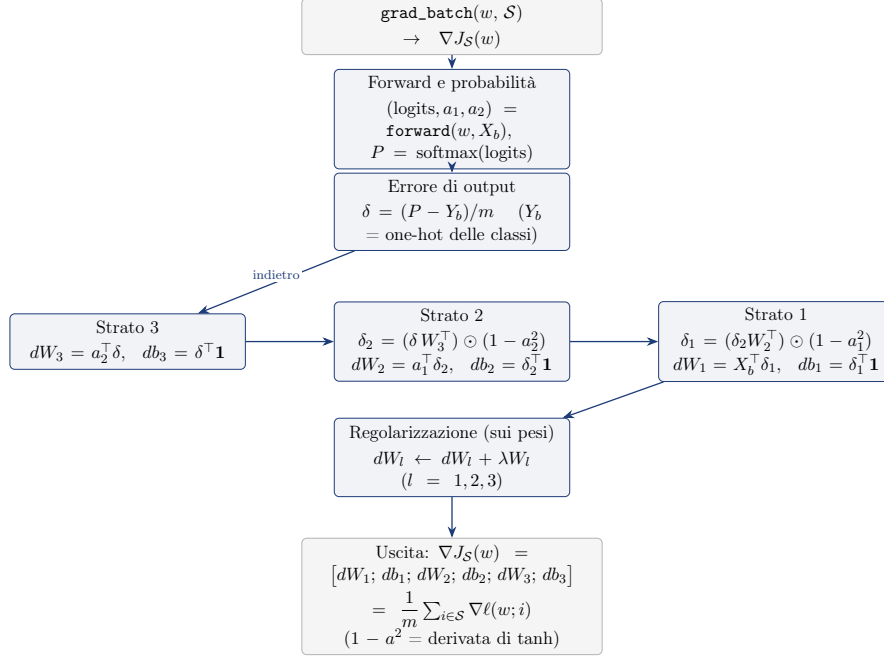


Figura 0.14: Funzione `grad_batch`: backpropagation vettorizzata. L'errore δ tra probabilità e one-hot è propagato all'indietro con la derivata di tanh ($1 - a^2$); a ogni matrice dei pesi si somma λW_l . Il risultato coincide con la media dei gradienti per-esempio del codice B.1.

Varianza per-esempio per la CCV. La condizione di controllo della varianza (codice B.1) richiede la varianza campionaria dei gradienti individuali sul batch corrente. In forma esatta essa vale $V = \frac{\sum_i \|g_i\|^2 - n_k \|\bar{g}\|^2}{n_k - 1}$: per calcolarla non serve materializzare la matrice ($n_k \times D$) dei gradienti individuali (che richiederebbe $n_k \cdot D$ valori, troppi per la memoria di Colab), basta accumulare la somma dei quadrati delle norme $\sum_i \|g_i\|^2$ calcolando i gradienti a blocchi di 32 esempi. Il valore ottenuto è *identico* a quello del codice B.1. La Figura 0.15 riassume la funzione `ccv_stats` e la sua dipendenza da `grad_batch` e `per_example_sq_norms`.

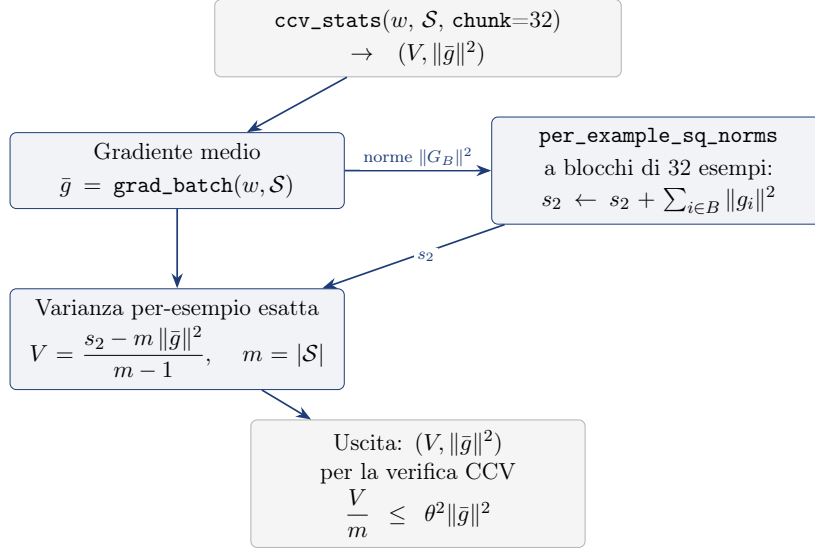


Figura 0.15: Funzione `ccv_stats`: gradiente medio dal `grad_batch`, somma dei quadrati delle norme per-esempio calcolata a blocchi di 32 esempi (per non materializzare la matrice $n_k \times D$) e combinazione nella formula esatta della varianza campionaria; l’uscita alimenta la verifica CCV del codice B.1.

Prodotto Hessiano–vettore. Il metodo Newton-CG richiede, a ogni iterazione del gradiente coniugato, il prodotto tra l’Hessiana della loss media sul sottocampione $\mathcal{H}_k$ e un vettore v . Poiché l’Hessiana della media è la media delle Hessiane, questo prodotto coincide con la media dei prodotti per-esempio; `hess_batch` lo calcola con un’unica chiamata di differenziazione automatica sulla funzione $f(w') = J_{\mathcal{H}_k}(w')$ (Figura 0.16). Per contenere il costo si pone $\gamma = 0$ nel criterio di arresto del CG (la stima della varianza degli Hessiani per-esempio sarebbe proibitiva) e si usa $\alpha = 1$ come passo iniziale delle line search, come nel resto della tesi.

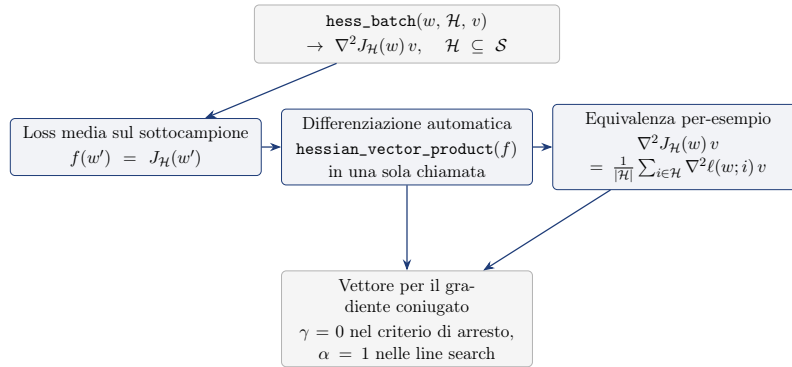


Figura 0.16: Funzione `hess_batch`: il prodotto Hessiano–vettore sul sottocampione $\mathcal{H}$ è calcolato definendo la loss media f e chiamando `hessian_vector_product` una sola volta; il valore coincide con la media dei prodotti per-esempio.

Il notebook `nsynth_net_riproduzione`, disponibile nella cartella `tesi/nsynth/` del repository, implementa fedelmente le funzioni qui descritte: la cella di validazione interna verifica numericamente, su un piccolo batch, che le funzioni batch coincidano con le versioni per-esempio (errori inferiori a 10^{-8} sul gradiente, sulla loss e sulla varianza CCV, e inferiori a 10^{-6} sul prodotto Hessiano–vettore), garantendo che l’uso delle versioni vettorizzate non altera i risultati degli algoritmi.

Risultati dell’esperimento. I quattro metodi sono stati applicati alla rete con 200 iterazioni, seed 42, batch iniziale 128 e soglia CCV $\theta = 0,3$; per contenere il costo delle iterazioni la dimensione del batch è stata limitata a $n_k \leq 2048$ (senza limite la CCV tenderebbe a far crescere il batch verso l’intero training set, $N = 12\,678$). La Tabella 0.2 riporta per ogni metodo l’accuratezza sul test set (4096 clip), la norma del gradiente finale, la dimensione del batch finale, il tempo di addestramento e, per il metodo con regolarizzazione L_1 , il numero di coefficienti non nulli. La Figura 0.17 mostra l’accuratezza di test al variare dell’iterazione k , mentre la Figura 0.18 mostra la dinamica della dimensione del batch n_k scelta dal controllo CCV.

Tabella 0.2: Rete neurale su NSynth: risultati dei quattro metodi (200 iterazioni, cap del batch a 2048, seed 42; tempi su Apple M5).

Metodo	Acc. test	$\ \nabla J(w)\ _2$	Batch fin.	Tempo (s)	Coeff. $\neq 0$
Dynamic GD	96.9%	4.1×10^{-1}	2048	148.8	—
Newton-CG	99.2%	3.0×10^{-2}	2048	210.2	—
Newton-CG L_1	97.3%	7.0×10^{-1}	2048	196.9	58 412/234 122
BB-CCV	96.2%	8.4×10^{-2}	2048	216.9	—

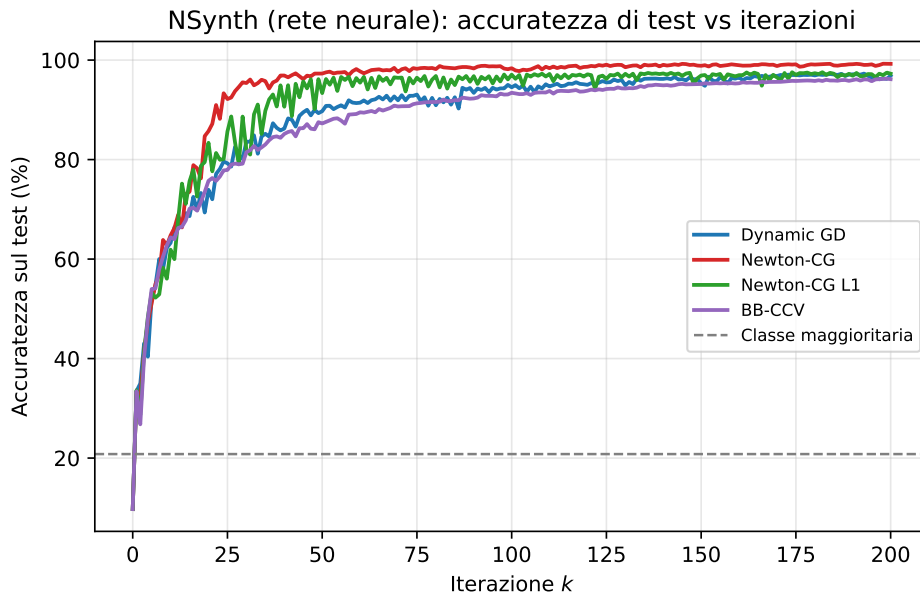


Figura 0.17: NSynth (rete neurale): accuratezza sul test set in funzione delle iterazioni per i quattro metodi; la linea tratteggiata indica la frequenza della classe maggioritaria (20.8%).

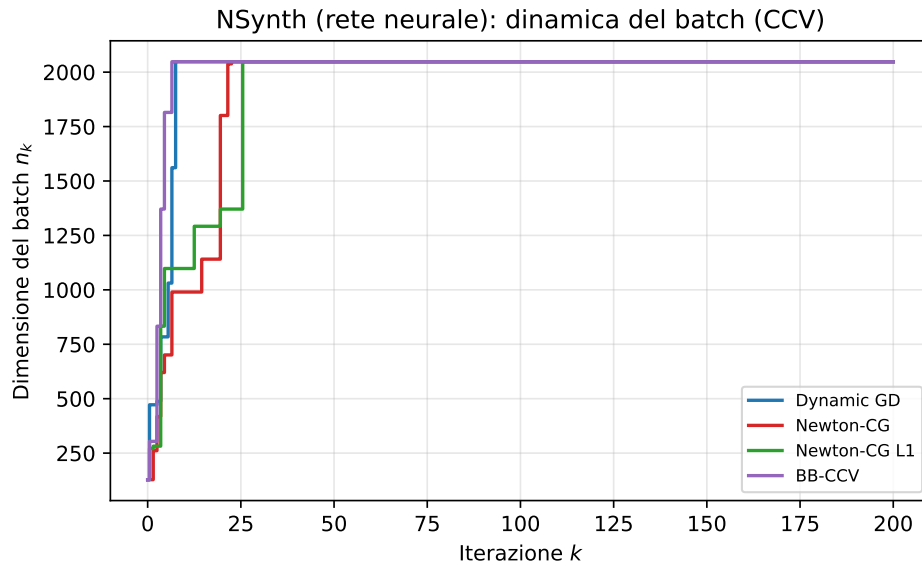


Figura 0.18: NSynth (rete neurale): dimensione del batch n_k scelta dal controllo CCV a ogni iterazione per i quattro metodi (tetto a 2048).

Il codice completo è disponibile nel notebook Colab `nsynth_net_riproduzione`, https://colab.research.google.com/github/Alessandro1040/selezione-dinamica-batch/blob/main/tesi/nsynth/nsynth_net_riproduzione.ipynb; un secondo notebook, `nsynth_net_test`, https://colab.research.google.com/github/Alessandro1040/selezione-dinamica-batch/blob/main/tesi/nsynth/nsynth_net_test.ipynb, carica i pesi addestrati e lo scaler e verifica i modelli su clip reali con le etichette vere: per una clip scelta (per indice o nome) stampa la famiglia vera e le predizioni dei quattro metodi — ad esempio la clip `keyboard_electronic_078-053-050` del test set è classificata come `keyboard` da tutti i metodi con probabilità superiore al 99% — e permette inoltre di caricare un proprio file audio e di ascoltare le clip. L'intero repository, con script, figure e note operative, è disponibile su <https://github.com/Alessandro1040/selezione-dinamica-batch>.

Riproducibilità. Tutti gli esperimenti di questa sezione (estrazione delle features, addestramento dei quattro metodi, figure e tabelle) sono riproducibili eseguendo dall'inizio il notebook Colab `nsynth_net_riproduzione` indicato sopra: le celle successive al download di NSynth (split validation e test) estraggono le features, eseguono i quattro metodi con i parametri dichiarati in questa sezione e generano le figure e la tabella in formato LaTeX con gli stessi valori qui riportati; la cella di validazione interna garantisce che le versioni vettorizzate delle funzioni non alterino i risultati rispetto alle versioni per-esempio dei codici B.1–B.4.

Precisione sui valori numerici. I dati riportati in questa sezione (accuratezze, norme del gradiente, dimensioni del batch e tempi) si riferiscono alla run di riferimento eseguita sul computer dell'autore, non a un'esecuzione in Colab. Il notebook Colab riproduce lo stesso codice con lo stesso seme casuale (seed 42), ma la riproducibilità non è esatta: gli algoritmi sono stocastici e i calcoli in virgola mobile dipendono dall'ambiente di esecuzione (versione di NumPy e libreria BLAS sottostante, OpenBLAS in Colab contro Accelerate su macOS), per cui piccole differenze di arrotondamento, accumulate sulle 200 iterazioni, producono valori finali leggermente diversi. Le differenze sono dell'ordine di qualche decimo di punto percentuale

e non cambiano le conclusioni; i tempi di esecuzione in Colab sono invece molto più lunghi (decine di minuti, contro i pochi minuti della macchina locale).

Riferimenti bibliografici

- [1] J. Engel, C. Resnick, A. Roberts, S. Dieleman, M. Norouzi, D. Eck, and K. Simonyan, “Neural audio synthesis of musical notes with WaveNet autoencoders,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.